ICMC USP SÃO CARLOS

USP Universidade de São Paulo



IA, Otimização e IoT para Indústrias e Negócios
Do Zero ao MLOps

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT

Módulo 1

Fundamentos de Dados e Aprendizado de Máquina

Aplicado a Operações e Negócios

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Construção de Pipelines e Prevenção de Vazamento de Dados

Sumário

01 O que é Pipeline e Vazamento de Dados?

02 Fluxo com e sem Pipeline

03 Usando Pipeline

04 Resumo

O que é Pipeline e Vazamento de Dados?

OpAIoT
Optimization • AI • IoT

1 **Fluxo manual**

Entender X, y, treino/teste, fit, transform, predict e métricas.

2 **Risco de erro**

Perceber onde pré-processamento mal aplicado causa avaliação otimista.

3 **Pipeline**

Organizar o fluxo para repetir as mesmas etapas em treino, teste e produção.

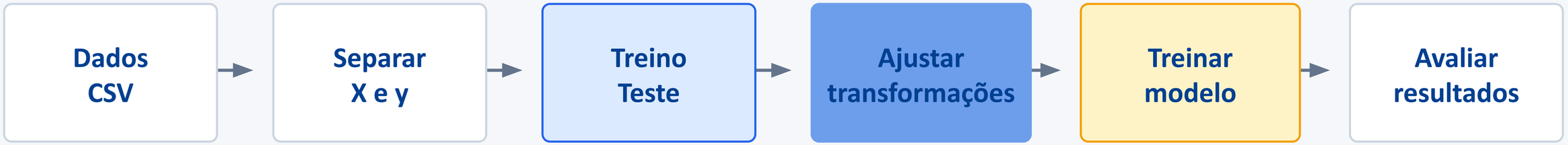
Pipeline é uma sequência organizada de etapas usadas para transformar dados brutos em um modelo treinado e avaliado.

- A principal vantagem do pipeline é garantir que as mesmas etapas aplicadas no treinamento sejam aplicadas corretamente aos dados futuros.

Vazamento de dados, ou data leakage, ocorre quando informações que não deveriam estar disponíveis no momento do treinamento ou da previsão acabam sendo usadas pelo modelo.

- Faz o modelo parecer melhor do que realmente é, porque ele aprende usando informação “indevida”.

Fluxo Manual Antes do Pipeline: entendendo o fluxo manual




```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

modelo.fit(X_train_scaled, y_train)
y_pred = modelo.predict(X_test_scaled)
```

O teste só pode passar por transformações aprendidas no treino.

Vazamento: Escalonar antes do split

```
# Errado: o scaler vê treino + teste
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.25, random_state=42
)
```

Por que é errado?

- A média e o desvio padrão são calculados também com dados de teste.
- O conjunto de teste deixa de simular dados realmente novos.
- A métrica pode ficar otimista e enganar a seleção de modelo.

⚠️ Sintoma típico: “o modelo parecia excelente na validação, mas piorou muito em produção”.

Toda a base → **fit_transform()** → **split depois** → **teste contaminado**

Fluxo correto sem Pipeline

```
# Correto: primeiro divide, depois ajusta no treino
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Memorizar!!!

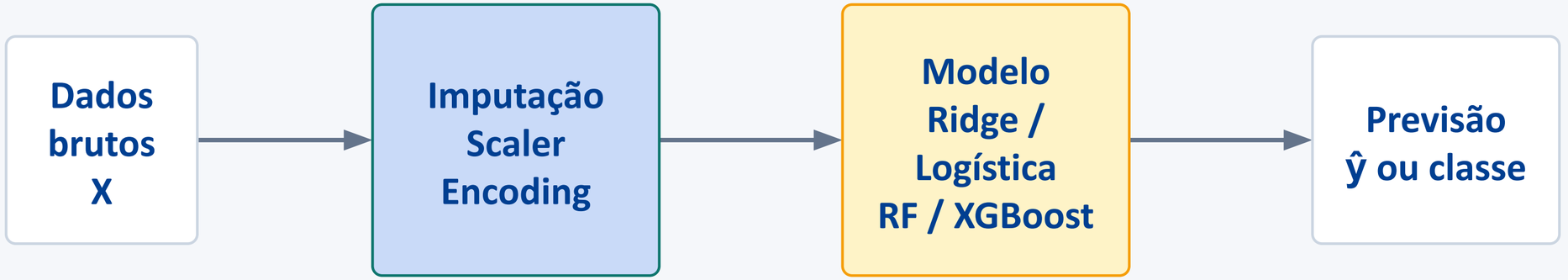
1. **fit**
   aprende parâmetros somente no treino

2. **transform**
   aplica parâmetros já aprendidos

3. **predict**
   gera previsões em dados já processados

No teste, nunca usamos fit. Usamos apenas transform e predict.

Fluxo correto com Pipeline



- Um Pipeline encadeia transformadores e um estimador final. O mesmo objeto aprende, transforma e prevê.
- Benefício principal: as transformações são ajustadas no treino e reaplicadas corretamente ao teste, validação cruzada e produção.

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipe = Pipeline([
    ("padronizacao", StandardScaler()),
    ("modelo", Ridge(alpha=1.0))
])

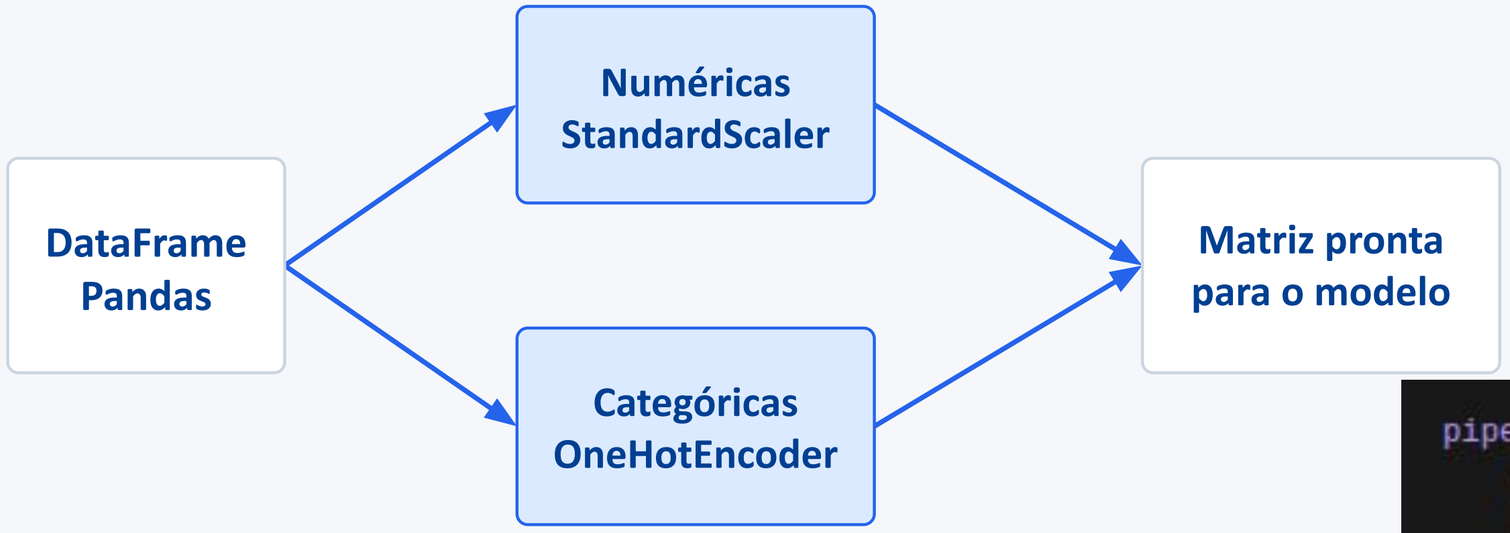
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

Anatomia de um Pipeline em scikit-learn

- Cada etapa tem um nome: "padronizacao" e "modelo".
- Todas as etapas antes da última precisam ter transform().
- A última etapa é o estimador: regressão, classificação ou outro modelo.
- O método fit() percorre as etapas na ordem definida.

ColumnTransformer: tratamentos diferentes por tipo de coluna

Bases têm colunas numéricas e categóricas. Cada grupo precisa de uma transformação adequada.




```
preprocessador = ColumnTransformer([
    ("num", StandardScaler(), atributos_numericos),
    ("cat", OneHotEncoder(handle_unknown="ignore"), atributos_categoricos)
])
```

```
pipe_reg = Pipeline([
    ("prep", preprocessador),
    ("modelo", Ridge(alpha=1.0))
])

pipe_reg.fit(X_train, y_train)
y_pred = pipe_reg.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

handle_unknown="ignore" evita erro se, no teste, aparecer uma categoria não vista no treino.

Validação cruzada: por que Pipeline é ainda mais importante?

Dobra 1	**teste**	**treino**	**treino**	**treino**	**treino**
Dobra 2	**treino**	**teste**	**treino**	**treino**	**treino**
Dobra 3	**treino**	**treino**	**teste**	**treino**	**treino**

Em cada dobra, o pré-processamento precisa ser ajustado apenas com a parte de treino daquela dobra.

```python
pipe_reg = Pipeline([
    ("prep", preprocessador),
    ("modelo", Ridge(alpha=1.0))
])

pipe_reg.fit(X_train, y_train)
y_pred = pipe_reg.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```python
scores = cross_val_score(
    pipe_reg,
    X,
    y,
    cv=5,
    scoring="neg_mean_absolute_error"
)
```

Com Pipeline, cada dobra ajusta scaler, encoder e modelo dentro do próprio treino da dobra.

Seleção de modelos e hiperparâmetros sem vazamento

Pipeline permite buscar parâmetros de transformação e modelo juntos, respeitando a validação.

```
param_grid = {
    "modelo__alpha": [0.01, 0.1, 1, 10]
}

busca = GridSearchCV(
    pipe_reg,
    param_grid=param_grid,
    cv=5,
    scoring="neg_mean_absolute_error"
)

busca.fit(X_train, y_train)
melhor_modelo = busca.best_estimator_
```

- O nome modelo__alpha acessa o parâmetro alpha dentro da etapa chamada modelo.
- GridSearchCV testa combinações usando validação cruzada.
- O teste final deve ser usado apenas uma vez, após escolher o melhor fluxo.
- Evite ajustar decisões olhando repetidamente o conjunto de teste.

⚠️ Mensagem: seleção de modelo também pode vazar informação se o teste for usado como validação repetidas vezes.

Árvores, florestas e XGBoost: precisa padronizar?

Nem todo modelo precisa das mesmas transformações. Pipeline organiza, mas não obriga a usar scaler.

1 Modelos lineares

Linear, Ridge, Lasso e Logística geralmente se beneficiam de padronização.

2 Árvores

Decision Tree não depende da escala das variáveis numéricas.

3 Ensembles de árvores

Random Forest e XGBoost também não exigem StandardScaler para variáveis numéricas.

```
preprocessador_arvores = ColumnTransformer([
    ("num", "passthrough", atributos_numericos),
    ("cat", OneHotEncoder(handle_unknown="ignore"), atributos_categoricos)
])

pipe_rf = Pipeline([
    ("prep", preprocessador_arvores),
    ("modelo", RandomForestClassifier(random_state=42))
])
```

“passthrough” mantém colunas numéricas sem transformação, mas ainda permite tratar categóricas com OneHotEncoder.

Resumo

1. Dividir treino/teste antes de aprender qualquer estatística dos dados.
2. Usar fit somente no treino; no teste, usar transform e predict.
3. Se criar alvo a partir de uma variável, remover essa variável de X.
4. Fazer seleção de atributos dentro da validação, não antes dela.
5. Usar Pipeline + ColumnTransformer quando houver várias etapas.
6. Reservar o conjunto de teste para a avaliação final.

O teste representa o futuro; o futuro não pode ensinar o passado.

split⟶ fit no treino⟶ transform no teste ⟶ predict⟶ avaliar

Se repetir esse fluxo, use Pipeline.

- ➔ **Pipeline não substitui entendimento: ele organiza um fluxo que o aluno já deve compreender.**
- ➔ **Data leakage torna a avaliação artificialmente otimista e compromete a escolha do modelo.**
- ➔ A regra principal é ajustar transformações apenas no treino.
- ➔ ColumnTransformer permite tratar colunas numéricas e categóricas no mesmo fluxo.
- ➔ Validação cruzada e busca de hiperparâmetros devem envolver o pipeline completo.

OpAIoT
Optimization • AI • IoT

Universidade de São Paulo

Referências

- FACELI, Katti; LORENA, Ana Carolina; GAMA, João; ALMEIDA, Tiago Agostinho de; CARVALHO, André Carlos Ponce de Leon Ferreira de. Inteligência artificial: uma abordagem de aprendizado de máquina. 2. ed. Rio de Janeiro: LTC, 2021.
- SCIKIT-LEARN DEVELOPERS. Pipelines and composite estimators. In: Scikit-learn User Guide. [S. l.]: scikit-learn, 2026. Disponível em: documentação oficial do scikit-learn. Acesso em: 10 maio 2026.
- SCIKIT-LEARN DEVELOPERS. Common pitfalls and recommended practices. In: Scikit-learn User Guide. [S. l.]: scikit-learn, 2026. Disponível em: documentação oficial do scikit-learn. Acesso em: 10 maio 2026.
- KAUFMAN, S.; ROSSET, S.; PERLICH, C.; STITELMAN, O. Leakage in data mining: formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data, New York, v. 6, n. 4, artigo 15, 2012. DOI: 10.1145/2382577.2382579.





Obrigado!

opaiot@icmc.usp.br
+55 (16) 99962-7637

PROFESSOR

Dr. Cláudio Fabiano Motta Toledo

OpAIoT
Optimization • AI • IoT